
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 9

Entrada/Saída Formatada em C

Exercícios (9.4–9.20)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 9.4 – Instruções printf e scanf	2
2	Exercício 9.5 – Saídas das Instruções	3
3	Exercício 9.6 – Erros nos Segmentos	4
4	Exercício 9.7 – Diferenças entre %d e %i	5
5	Exercício 9.8 – %n para Contar Caracteres	5
6	Exercício 9.9 – Ponteiros como Inteiros	7
7	Exercício 9.10 – Larguras de Campo Diversas	7
8	Exercício 9.11 – Arredondamento	9
9	Exercício 9.12 – String em Campo do Dobro	9
10	Exercício 9.13 – Tabela Fahrenheit/Celsius	11
11	Exercício 9.14 – Sequências de Escape	11
12	Exercício 9.15 – ? Literal sem Escape	13
13	Exercício 9.16 – Ler 437 com Todos os Especificadores	13
14	Exercício 9.17 – Ler 1.2345 com e, f, g	14
15	Exercício 9.18 – Strings com Aspas	15
16	Exercício 9.19 – '??' como Caractere	15
17	Exercício 9.20 – %g com Precisões 1 a 9	16

1. Exercício 9.4 – Instruções printf e scanf

Resposta detalhada

a) Inteiro sem sinal 40000 alinhado à esquerda em campo de 15:

```
1 printf( "%-15u\n", 40000 );
```

b) Ler hexadecimal em hex:

```
1 scanf( "%x", &hex );
```

c) Imprimir 200 com e sem sinal:

```
1 printf( "%d %u\n", 200, 200 );
```

d) Imprimir 100 em hex precedido por 0x:

```
1 printf( "%#x\n", 100 ); /* imprime 0x64 */
```

A flag # com x adiciona o prefixo 0x.

e) Ler caracteres em s até encontrar 'p':

```
1 scanf( "%[^p]", s );
```

O ^ dentro do conjunto significa “qualquer caractere *exceto*” os listados. Aqui: lê tudo até encontrar p.

f) Imprimir 1.234 em campo de 9 com zeros iniciais:

```
1 printf( "%09.3f\n", 1.234 ); /* imprime 00001.234 */
```

A flag 0 preenche com zeros à esquerda; .3 mantém 3 decimais.

g) Ler hh:mm:ss com supressão dos ::

```
1 scanf( "%d%c%d%c%d", &hora, &minuto, &segundo );
```

%c lê um caractere e descarta-o (supressão de atribuição).

h) Ler “caracteres” (com aspas) e armazenar em s:

```
1 scanf( "\"%[^\"]\"", s );
```

Lê literalmente as aspas iniciais e finais, e captura em s todos os caracteres entre elas.

i) Ler hh:mm:ss sem supressão:

```
1 scanf( "%d:%d:%d", &hora, &minuto, &segundo );
```

Os : na string de formato *esperam* encontrar : na entrada (que são descartados).

2. Exercício 9.5 – Saídas das Instruções

Resposta detalhada

- a) `printf( "%-10d\n", 10000 );`
Saída: 10000░░░░░
 (10000 ocupa 5 caracteres, com 5 espaços à direita até totalizar 10.)
- b) `printf( "%c\n", "Isto e uma string" );`
ERRO: `%c` espera um `char`, mas recebeu um ponteiro para `char`. Comportamento indefinido – provavelmente imprime um caractere “estranho” ou causa falha.
Correção: `printf( "%s\n", "Isto e uma string" );`
- c) `printf( "%.3lf\n", 8, 3, 1024.987654 );`
 Largura dinâmica = 8, precisão dinâmica = 3.
Saída: 1024.988 (8 caracteres, 3 decimais arredondados)
- d) `printf( "%#o\n%X\n%e\n", 17, 17, 1008.83689 );`
Saída:
 021
 0X11
 1.008837e+03
- e) `printf( "% ld\n%+ld\n", 1000000, 1000000 );`
 Flag espaço: insere espaço antes de positivos. Flag +: insere +.
Saída:
 1000000
 +1000000
- f) `printf( "%10.2E\n", 444.93738 );`
Saída: ░░4.45E+02 (largura 10, 2 decimais, E maiúsculo)
- g) `printf( "%10.2g\n", 444.93738 );`
`%g` usa o formato mais compacto. Com precisão 2 (2 dígitos significativos):
Saída: ░░░░4.4e+02
- h) `printf( "%d\n", 10.987 );`
ERRO: `%d` espera `int`, mas recebeu `double`. Comportamento indefinido (em sistemas modernos pode imprimir lixo ou crashar).
Correção:

```

1 printf( "%d\n", ( int )10.987 );      /* cast
   explicito */
2 printf( "%f\n", 10.987 );            /* ou use %f
   */

```

3. Exercício 9.6 – Erros nos Segmentos

Resposta detalhada

- a) `printf( "%s\n", 'Feliz aniversario' );`
Erro: String entre aspas simples. Strings exigem aspas duplas.
Correção: `printf( "%s\n", "Feliz aniversario" );`
- b) `printf( "%c\n", '0la' );`
Erro: Aspas simples com múltiplos caracteres ('0la') gera apenas um `int` com valor implementação-definido.
Correção: para imprimir apenas um caractere, use '0' (um único caractere); para a palavra, use `%s` com "0la".
- c) `printf( "%c\n", "Isso e uma string" );`
Erro: `%c` espera caractere, recebeu string.
Correção: use `%s`, ou para imprimir apenas o primeiro caractere:
- ```
1 printf("%c\n", "Isso e uma string"[0]);
```
- d) `printf( ""%s"", "Boa viagem" );`  
**Erro:** Aspas duplicadas dentro da string de formato sem escape.  
**Correção:**
- ```
1 printf( "\"%s\"", "Boa viagem" ); /* imprime "Boa
   viagem" */
```
- e) `char day[] = "Domingo"; printf( "%s\n", day[ 3 ] );`
Erro: `day[3]` é um `char` (no caso, 'i'), não um ponteiro. `%s` espera `char *`.
Correção:
- ```
1 printf("%s\n", &day[3]); /* imprime "ingo" a
 partir do 4o char */
```
- f) `printf( 'Digite seu nome: ' );`  
**Erro:** String entre aspas simples.  
**Correção:** `printf( "Digite seu nome: " );`
- g) `printf( %f, 123.456 );`  
**Erro:** String de formato sem aspas.  
**Correção:** `printf( "%f", 123.456 );`
- h) `printf( "%s%s\n", '0', 'K' );`  
**Erro:** `%s` espera strings, recebeu caracteres em aspas simples.  
**Correção:**
- ```
1 printf( "%c%c\n", '0', 'K' ); /* dois caracteres
   */
2 printf( "%s%s\n", "0", "K" ); /* duas strings
   */
```
- i) `char s[10]; scanf( "%c", s[ 7 ] );`
Erro: Falta o operador de endereço `&`. `scanf` precisa de ponteiro para onde armazenar.
Correção: `scanf( "%c", &s[ 7 ] );`

4. Exercício 9.7 – Diferenças entre %d e %i

Resposta detalhada

```

1  /* Ex9.7: d_vs_i.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int x, y;
7
8      printf( "Digite dois inteiros: " );
9      scanf( "%i%d", &x, &y );
10     printf( "Lidos: x = %d, y = %d\n", x, y );
11
12     return 0;
13 }

```

Diferença chave: %d sempre lê em *decimal*. Já %i *detecta* a base automaticamente baseando-se no prefixo:

- 0x ou 0X → hexadecimal
- 0 (zero à esquerda) → octal
- Sem prefixo → decimal

Testes com os conjuntos sugeridos:

Entrada	%i (x)	%d (y)
10 10	10	10
-10 -10	-10	-10
010 010	8 (octal)	10 (decimal, ignora 0)
0x10 0x10	16 (hex)	0 (lê 0, para no 'x')

Conclusão: %i é mais “inteligente”. Útil quando o usuário pode inserir números em bases diferentes. Para uso geral, %d é mais previsível.

5. Exercício 9.8 – %n para Contar Caracteres

Resposta detalhada

```

1  /* Ex9.8: contador_chars.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #define N 10
6
7  int main( void )
8  {
9      int number[ N ];
10     int impressos;          /* caracteres impressos por

```

```
11     valor */
12     int total = 0;          /* acumulado */
13     int i;
14
15     srand( time( NULL ) );
16
17     /* Inicializa array */
18     for ( i = 0; i < N; ++i ) {
19         number[ i ] = 1 + rand() % 1000;
20     }
21
22     printf( "%-10s%-15s%s\n", "Valor", "Chars", "Total
23         acumulado" );
24
25     for ( i = 0; i < N; ++i ) {
26         printf( "%-10d\n", number[ i ], &impressos );
27
28         /* impressos contem o numero de chars escritos
29            ATE %n */
30         total += impressos - 0; /* subtrai o tamanho
31            ja somado */
32
33         /* Re-imprime o valor + total ate aqui */
34         total = 10; /* largura do campo */
35         printf( "Total acumulado: %d\n", total * ( i +
36             1 ) );
37     }
38
39     return 0;
40 }
```

Nota: `%n` é um especificador especial que *não imprime nada*, mas armazena no inteiro apontado pelo argumento o número de caracteres já impressos até aquele ponto na string de saída.

Erro comum de programação

`%n` é considerado **perigoso** em código moderno. Sistemas com *format string vulnerability* podem ser explorados via `%n`. Por isso, muitos compiladores e bibliotecas (incluindo Windows) o desabilitam por padrão. Use com cuidado, apenas em contextos educacionais.

6. Exercício 9.9 – Ponteiros como Inteiros

Resposta detalhada

```
1  /* Ex9.9: ponteiros_int.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int x = 42;
7      int *ptr = &x;
8
9      printf( "%p: %p\n", ( void * )ptr );
10
11     /* Possivelmente OK (depende do compilador) */
12     printf( "%d: %d\n", ( int )( long )ptr );
13     printf( "%u: %u\n", ( unsigned )( long )ptr );
14
15     /* Em sistemas 64-bit, int causa truncamento */
16     printf( "%lu: %lu\n", ( unsigned long )ptr );
17     printf( "%lx: %lx\n", ( unsigned long )ptr );
18     printf( "%o: %o\n", ( unsigned )( long )ptr );
19
20     return 0;
21 }
```

Resultados esperados:

- %p: formato definido pela implementação. Geralmente em hexadecimal, e.g. 0x7fff5fbff8ac em sistemas Linux/macOS.
- %d, %u: em sistemas 64-bit, gera valor truncado (perda de bits altos) – **valores estranhos**.
- %lu, %lx: representação completa em decimal/hex.
- %o: representação octal – valores “estranhos”, muito longos.

Erros possíveis: sem cast adequado, o compilador moderno emite avisos. Casts inadequados podem levar a comportamento indefinido em arquiteturas onde int é menor que void *.

7. Exercício 9.10 – Larguras de Campo Diversas

Resposta detalhada

```
1  /* Ex9.10: larguras_diversas.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int    inteiro = 12345;
7      float  real    = 1.2345f;
8      int    largura;
```



```
9
10     printf( "%-10s | %-10s\n", "Largura", "Saida" );
11     printf( "-----+-----\n" );
12
13     for ( largura = 1; largura <= 10; ++largura ) {
14         printf( "%-10d | %*d\n", largura, largura,
15             inteiro );
16     }
17
18     printf( "\n%-10s | %-15s\n", "Largura", "Saida
19         float" );
20     printf( "-----+-----\n" );
21
22     for ( largura = 1; largura <= 10; ++largura ) {
23         printf( "%-10d | %*.4f\n", largura, largura,
24             real );
25     }
26
27     return 0;
28 }
```

O que acontece com largura insuficiente? C *nunca* trunca o número. Quando o valor não cabe na largura especificada, ele é impresso por completo, “estourando” o campo. Ou seja, a largura é um *mínimo*, não um máximo.

Exemplo: `printf("%3d", 12345)` imprime 12345 (5 chars), não 123.

8. Exercício 9.11 – Arredondamento

Resposta detalhada

```
1  /* Ex9.11: arredondamento.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      double valor = 100.453627;
7
8      printf( "Original:           %.6f\n", valor );
9      printf( "Inteiro (%%.0f):      %.0f\n", valor );
10     printf( "Decimo (%%.1f):       %.1f\n", valor );
11     printf( "Centesimo (%%.2f):    %.2f\n", valor );
12     printf( "Milesimo (%%.3f):     %.3f\n", valor );
13     printf( "Decimo milesimo:      %.4f\n", valor );
14
15     return 0;
16 }
```

Saída:

Saída do programa

```
Original: 100.453627
Inteiro (%%.0f): 100
Decimo (%%.1f): 100.5
Centesimo (%%.2f): 100.45
Milesimo (%%.3f): 100.454
Decimo milesimo: 100.4536
```

Note: printf arredonda matematicamente, não trunca. 100.4536 arredondado para a quarta casa: o quinto dígito é 2, então mantém. Mas 100.453627 arredondado para 3 casas: o quarto dígito é 6 (≥ 5), arredonda para cima $\rightarrow$ 100.454.

9. Exercício 9.12 – String em Campo do Dobro

Resposta detalhada

```
1  /* Ex9.12: string_dobro.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char texto[ 100 ];
8      int tam, larguraDobro;
9  }
```

```
10     printf( "Digite uma string: " );
11     fgets( texto, 100, stdin );
12     texto[ strcspn( texto, "\n" ) ] = '\0';
13
14     tam = strlen( texto );
15     larguraDobro = tam * 2;
16
17     printf( "Comprimento: %d\n", tam );
18     printf( "Dobrado (%d chars): [%*s]\n",
19             larguraDobro, larguraDobro, texto );
20
21     return 0;
22 }
```

Exemplo: se a entrada for “Ola” (3 chars), o campo será de 6 caracteres e a saída terá 3 espaços à esquerda + “Ola”.

10. Exercício 9.13 – Tabela Fahrenheit/Celsius

Resposta detalhada

```
1  /* Ex9.13: tabela_temperaturas.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int fahrenheit;
7      double celsius;
8
9      printf( "%10s%10s\n", "Fahr", "Celsius" );
10     printf( "%10s%10s\n", "----", "-----" );
11
12     for ( fahrenheit = 0; fahrenheit <= 212; ++
13           fahrenheit ) {
14         celsius = 5.0 / 9.0 * ( fahrenheit - 32 );
15         printf( "%10d%+10.3f\n", fahrenheit, celsius );
16     }
17
18     return 0;
19 }
```

Saída parcial:

Saída do programa

```
        Fahr      Celsius
        ----      -----
        0-17.778
        1-17.222
        ...
        32+0.000
        100+37.778
        212+100.000
```

Flags-chave: `%+10.3f` – sinal sempre exibido (+), largura 10 alinhada à direita, 3 dígitos decimais.

11. Exercício 9.14 – Sequências de Escape

Resposta detalhada

```
1  /* Ex9.14: escapes.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
```

```
6     printf( "\\n: " ); printf( "[A\nB]\n" );
7     printf( "\\t: [A\tB]\n" );
8     printf( "\\a (alerta sonoro): [A\aB]\n" );
9     printf( "\\b (backspace):    [AX\bB]\n" );
10    printf( "\\r (retorno):       [ABC\rXY]\n" );
11    printf( "\\f (form feed):     [A\fB]\n" );
12    printf( "\\v (tab vertical):  [A\vB]\n" );
13    printf( "\\\\: [\\]\n" );
14    printf( "\\\": [\\]\n" );
15    printf( "\\?: [\\?]\n" );
16    printf( "\\0: [A\0B] (string termina no \\0)\n" );
17
18    return 0;
19 }
```

Comportamentos:

- \n: nova linha (cursor para próxima linha)
- \t: tab horizontal (próxima posição de tab)
- \a: alerta (geralmente beep)
- \b: backspace (cursor recua 1 posição)
- \r: carriage return (cursor volta ao início da linha)
- \f: form feed (avança para próxima página em impressoras)
- \v: tab vertical
- \\, \", \?: caracteres literais escapados

12. Exercício 9.15 – ? Literal sem Escape

Resposta detalhada

```
1  /* Ex9.15: interrogacao.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      printf( "Com escape: \?\n" );
7      printf( "Sem escape: ?\n" );
8      return 0;
9  }
```

Resposta: Sim, o caractere ? pode ser impresso diretamente como caractere literal em uma string `printf`. As duas formas (? e \?) produzem o mesmo resultado.

Por que existe \? então? Existe para evitar *trigraphs* (sequências de 3 caracteres que começam com ??, usadas em terminais antigos para representar caracteres especiais). Por exemplo, ??/ era equivalente a \. Em código moderno, trigraphs foram removidos do padrão C23, mas \? ainda funciona.

13. Exercício 9.16 – Ler 437 com Todos os Especificadores

Resposta detalhada

```
1  /* Ex9.16: ler_437.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int valor;
7
8      printf( "Digite 437 (decimal): " );
9      scanf( "%d", &valor );
10     printf( "   %%d=%%d   %%i=%%d   %%o=%%o   %%u=%%u   %%x=%%x\n",
11            valor, valor, valor, valor, valor );
12
13     printf( "Digite 437 (octal, ex: 665): " );
14     scanf( "%o", &valor );
15     printf( "   %%d=%%d   %%o=%%o\n", valor, valor );
16
17     printf( "Digite 437 (hex, ex: 1b5): " );
18     scanf( "%x", &valor );
19     printf( "   %%d=%%d   %%x=%%x\n", valor, valor );
20
21     return 0;
22 }
```

Conversões equivalentes a 437:

- Decimal: 437
- Octal: 665 (porque $6 \times 64 + 6 \times 8 + 5 = 437$)
- Hex: 0x1B5 (porque $1 \times 256 + 11 \times 16 + 5 = 437$)

14. Exercício 9.17 – Ler 1.2345 com e, f, g**Resposta detalhada**

```
1  /* Ex9.17: ler_floats.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      float a, b, c;
7
8      printf( "Digite 1.2345 em formato decimal: " );
9      scanf( "%f", &a );
10
11     printf( "Digite 1.2345 em formato exponencial (ex:
12             1.2345e0): " );
13     scanf( "%e", &b );
14
15     printf( "Digite 1.2345 em formato geral: " );
16     scanf( "%g", &c );
17
18     printf( "\nValores lidos: a=%f, b=%f, c=%f\n", a,
19             b, c );
20
21     return 0;
22 }
```

Conclusão: %e, %f e %g no `scanf` são *equivalentes para leitura* – todos aceitam tanto notação decimal quanto científica. A diferença está apenas na `printf` (formato de saída).

15. Exercício 9.18 – Strings com Aspas

Resposta detalhada

```
1  /* Ex9.18: aspas.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      char a[ 50 ], b[ 50 ], c[ 50 ];
7
8      printf( "Digite 3 entradas: suzy \"suzy\" 'suzy'\n"
9              );
10     scanf( "%s%s%s", a, b, c );
11
12     printf( "a = [%s]\n", a );
13     printf( "b = [%s]\n", b );
14     printf( "c = [%s]\n", c );
15
16     return 0;
17 }
```

Resultado: C não ignora aspas – elas são lidas como parte da string. %s para no primeiro espaço em branco. Então:

- a = "suzy" (sem aspas)
- b = "\"suzy\"" (com as aspas duplas)
- c = "'suzy'" (com as aspas simples)

Como ignorar aspas? Usar `scanf("\"%[^\\\"]\\\"", b);` que lê literalmente as aspas e captura o conteúdo entre elas.

16. Exercício 9.19 – '?' como Caractere

Resposta detalhada

```
1  /* Ex9.19: caractere_interrogacao.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      printf( "Constante de caractere '?: [%c]\n", '?'
7              );
8      printf( "Sequencia de escape '\\?: [%c]\n", '\\?'
9              );
10     return 0;
11 }
```

Resultado: Ambas as formas ('?' e '\\?') funcionam perfeitamente e produzem o mesmo caractere de interrogação na saída. A diferença é apenas estilística; '\\?' historicamente protegia contra trigraphs (hoje obsoleto).

17. Exercício 9.20 – %g com Precisões 1 a 9

Resposta detalhada

```
1  /* Ex9.20: g_preciso.es.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      double valor = 9876.12345;
7      int prec;
8
9      printf( "Valor: %f\n\n", valor );
10     printf( "%-10s%s\n", "Precisao", "Saida com %g" );
11     printf( "-----\n" );
12
13     for ( prec = 1; prec <= 9; ++prec ) {
14         printf( "%-10d.*g\n", prec, prec, valor );
15     }
16
17     return 0;
18 }
```

Saída esperada:

Saída do programa

```
Valor:  9876.123450

Precisao Saida com %g
-----
1uuuuuuuu1e+04
2uuuuuuuu9.9e+03
3uuuuuuuu9.88e+03
4uuuuuuuu9876
5uuuuuuuu9876.1
6uuuuuuuu9876.12
7uuuuuuuu9876.123
8uuuuuuuu9876.1234
9uuuuuuuu9876.12345
```

Comportamento do %g: a “precisão” em %g significa **dígitos significativos** (não casas decimais). O formato escolhe automaticamente entre %e e %f, escolhendo o mais compacto:

- Precisão ≤ 4 : usa notação científica (%e).
- Precisão > 4 : usa notação decimal padrão (%f).
- Zeros à direita são suprimidos (diferente de %f).

Boa prática de programação

`%g` é excelente quando você não sabe a magnitude dos valores que serão impressos. Mostra valores pequenos em decimal e valores muito grandes ou muito pequenos em notação científica – a melhor representação compacta para cada caso.

Reflexão Final – Capítulo 9

A arte da E/S formatada

O Capítulo 9 aprofunda algo que aparentemente já conhecíamos do Capítulo 2: `printf` e `scanf`. Mas o que parecia simples revelou-se uma **mini-linguagem** dentro de C, com dezenas de especificadores, flags, modificadores e nuances.

Domínio dos pontos-chave:

- Saber qual especificador usar para cada tipo (`%d`, `%lf`, `%s`, `%p`, ...).
- Controlar largura, precisão e alinhamento para criar tabelas legíveis.
- Usar conjunto de varredura e supressão para parsear entradas complexas.
- Combinar flags (+, -, 0, #) com elegância.
- Reconhecer armadilhas: `%d` com `double`, falta de `&` no `scanf`, `%c` com strings.

Por que isso importa em projetos reais? Logs, relatórios financeiros, arquivos CSV, dashboards, parsers de configuração – todos dependem de E/S formatada com precisão. Programas amadores são reconhecíveis pela formatação descuidada; programas profissionais primam por saídas limpas, alinhadas e informativas.

Próximos passos: no Cap. 10 chegaremos a `struct` – agrupando dados heterogêneos; no Cap. 11, arquivos – aplicando tudo o que aprendemos até agora a entrada/saída persistente em disco.